



PSC FINAL REPORT - INF 10 KNOWLEDGE TRACING ASSISTANT FOR LEARNING PROGRAMMING

24 April 2026

Alan ANDRÉ, Alexandre ITTAH, Abdelkayoum KADDOURI,
Mounir LBATH, Alexandre PARÉSY



CONTENTS

Introduction	3
0.1 Motivation and scope	3
1 Knowledge Tracing	5
1.1 Classical knowledge tracing models	5
1.1.1 Early and interpretable approaches	5
1.1.2 Attentive Knowledge tracing	6
1.2 Live knowledge tracing with tabular foundation models	10
1.2.1 Motivation	10
1.2.2 Related Work	10
1.2.3 Tabular Foundation Models for Live Knowledge Tracing	11
1.2.4 Experiments	11
1.2.5 Results	13
1.2.6 Discussion and Conclusion	13
2 Dynamic cognitive diagnosis	15
2.1 Mathematical Formulation	16
2.2 First Approach: Chain Decomposition and Binary Search	17
2.3 Second Approach: Theoretical Bound via the Central Element Theorem	19
2.4 Practical implementation: Finding the Central Element	22
3 Knowledge Mapping and Practical Applications	25
3.1 Domain representation	25
3.1.1 Competitive programming domain visualization	26
3.1.2 Dependency graph generation	26
3.2 Knowledge graph from an LLM	29
3.3 CFTracker : Knowledge state analysis and problem recommendation	31
3.3.1 Exploratory recommendation baselines	32
3.3.2 Final recommendation model: augmented Elo	34
4 Project Evaluation and Progress Review	35
4.1 Achievement of objectives	35
4.2 Further exploration possibilities	35
4.3 Exposure to research	36
4.4 Groupwork	36
Conclusion	37
Conclusion	37

INTRODUCTION

This report presents the work we carried out as part of the PSC, a year-long collective research project at the intersection of artificial intelligence, machine learning, and education. From the outset, our objective was to investigate how machine learning methods could be used to support the learning of programming, with a particular focus on competitive programming as a rich and structured educational setting.

We would like to express our sincere gratitude to Prof. Jill-Jênn Vie, researcher at INRIA and our tutor, for his invaluable guidance and support throughout the year. His expertise and mentorship allowed us to approach this topic with confidence and ultimately led us to submit a preprint to a major conference. We also thank Prof. Gilles Schaeffer, our coordinator, as well as the computer science department of École Polytechnique for making this project possible.

0.1 MOTIVATION AND SCOPE

Our project is rooted in the broader question of how to build intelligent tutoring systems for complex domains. In particular, we focus on knowledge tracing, which is the process of modeling a learner’s current state of skills and updating it as they progress. This allows for the creation of intelligent learning systems, and while we chose to narrow our focus on the domain of competitive programming, the methods presented in this work could be generalized to other fields.

There are two reasons why we chose to apply our work in the domain of competitive programming. Firstly, all 5 members of our team are familiar with algorithmic and programming problems, and three members are familiar with competitive programming as they took courses or went to competitions such as ICPC SWERC, the Southwestern Europe Regional Championship in the world’s largest university programming competition, while being coached by our tutor. Secondly, there are many competitive programming datasets and resources for machine learning on the internet, as this subject has become very popular for the purpose of training Large Language Models (LLMs) to generate code. We show in Figure 1 an example of a competitive programming problem.

Over the course of the PSC, our work developed along the three directions announced in our initial proposal. First, we studied and implemented several approaches to knowledge tracing, ranging from classical models such as Elo to modern deep learning methods such as Attentive Knowledge Tracing (AKT). This first exploration led us to a more original contribution: the formulation of a *live knowledge tracing* setting, motivated by contest-like environments where interactions must be processed online, and the development of a lightweight approach based on tabular foundation models. This work resulted in a paper accepted at AIED 2026. Second, we investigated how to represent the structure of knowledge in competitive programming. This

Problem F

Factor-Free Tree

Time limit: 6 seconds

A *factor-free tree* is a rooted binary tree where every node in the tree contains a positive integer value that is coprime with all of the values of its ancestors. Two positive integers are coprime if their greatest common divisor equals 1.

The *inorder sequence* of a rooted binary tree can be generated recursively by traversing first the left subtree, then the root, then the right subtree. See Figure F.1 below for the inorder sequence of one factor-free tree.

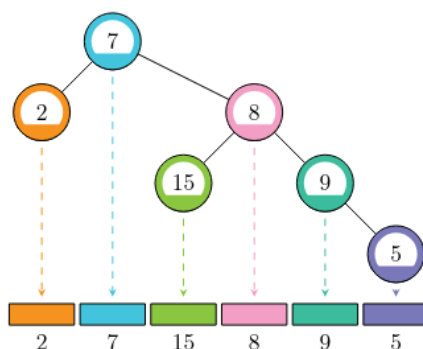


Figure F.1: Illustration of Sample 1. The tree is factor-free; for example, the value of the node marked “5” is coprime with all of the values of its ancestors, marked “9”, “8”, and “7”.

Given a sequence $a_1, a_2, \dots, a_n$, decide if it is the inorder sequence of some factor-free tree and if so construct such a tree.

Figure 1: Example of a competitive programming problem. Source : ICPC Northwest Europe Regional Championship 2017

led us to study prerequisite graphs and their mathematical formulation as partially ordered sets, and to derive algorithms for inferring a learner’s hidden knowledge state from a small number of queries. Third, we explored how such representations could be used in practice, both through the construction of knowledge graphs from tags and large language models, and through the development of a web application, CFTracker, designed to analyze a user’s profile and recommend suitable problems.

More broadly, this PSC gave us the opportunity to experience research in a concrete way. What began as a general interest in intelligent tutoring systems gradually evolved into a project combining literature review, mathematical modeling, machine learning experiments, algorithm design, and software development. The final report therefore reflects both the diversity of the methods we explored and the coherence of the central question that guided us throughout the year: how can we model and support the learning of programming in a principled, data-driven, and practically useful way?

1

KNOWLEDGE TRACING

The first objective of our PSC was to understand how a learner's knowledge can be modeled from sequences of interactions. This problem is known as *knowledge tracing*. As a first step, we studied the main approaches from the literature and implemented Attentive Knowledge Tracing (AKT) from scratch. Building on this work, we then introduced a new setting, called *LiveKT*, motivated by online and contest-like environments in which interactions arrive sequentially and predictions must be updated in real time. This work resulted in a paper accepted at AIED 2026.

1.1 CLASSICAL KNOWLEDGE TRACING MODELS

Definition 1.1 (Knowledge tracing (KT)) *Knowledge Tracing is the process of modeling a student's knowledge over time as they interact with learning activities.*

This information can be used in many ways, such as personalizing learning and recommending better resources. There is a large diversity of models for KT, but most of them rely on:

Definition 1.2 (Knowledge component (KC)) *A knowledge component is a small piece of knowledge or skill that a learner needs to master to solve a problem or complete a task.*

They are used to divide complex subjects into smaller parts that can be taught, practiced, and assessed. For example, in competitive programming, depth-first search on trees, prime numbers and the union-find data structure are 3 different KCs.

In the literature, most Knowledge Tracing models fall into one of the following three categories: Bayesian models, logistic models, and deep learning models [1].

1.1.1 • EARLY AND INTERPRETABLE APPROACHES

One of the earliest models for knowledge tracing is ***Bayesian Knowledge Tracing*** (BKT) [1]. It models each knowledge component (KC) independently as a latent binary variable evolving over time : either mastered or not mastered. Despite strong assumptions such as KC independence and binary mastery, BKT remains popular due to its simplicity and interpretability.

Instead of representing a learner's knowledge as a hidden binary state, ***logistic models*** estimate the probability of a correct response directly from observable features. For instance, in ***Performance Factor Analysis*** (PFA) [2], the probability of answering correctly the item

for student i is given by:

$$\mathbb{P}(\text{correct}_i) = \sigma\left(\beta + \sum_{j \in KCs} (\gamma_j s_{ij} + \rho_j f_{ij})\right)$$

$\sigma(x)$: logistic function $\frac{1}{1 + e^{-x}}$
 β : global bias term
 KCs : set of KCs in the exercise
 γ_j : learning gain weight per success for KC j
 ρ_j : penalty weight per failure for KC j
 s_{ij} : prior successes of student i on KC j
 f_{ij} : prior failures of student i on KC j

In such a method, parameters are typically estimated offline.

In contrast, the **Elo-based** approach [3] updates parameters online. For Elo, the probability that student i correctly solves a KC j is given by

$$\mathbb{P}(\text{correct}_{i,j}) = \sigma(\theta_i - \beta_j)$$

where θ_i is the student's mastery (usually called the student's Elo), β_j the KC difficulty (usually the problem's Elo), and $\sigma(x) = \frac{1}{1+e^{-x}}$. After each response, parameters are updated with

$$\theta_i \leftarrow \theta_i + K \cdot (r - \mathbb{P}(\text{correct}_{i,j})), \quad \beta_j \leftarrow \beta_j - K \cdot (r - \mathbb{P}(\text{correct}_{i,j}))$$

where $K > 0$ is the learning rate and $r \in \{0, 1\}$ stores whether the student solved the problem or not. Elo knowledge tracing is fast and adaptive, but ignores long-term practice history, unlike PFA. Elo is very popular in chess or competitive programming websites like Codeforces.

1.1.2 • ATTENTIVE KNOWLEDGE TRACING

While Bayesian and logistic models are attractive because of their interpretability, they often struggle to capture the complex temporal dependencies that arise in realistic learning sequences. This motivated the development of deep learning approaches to knowledge tracing. For example, **Deep Knowledge Tracing** (DKT) [4] uses recurrent neural networks.

Attentive Knowledge Tracing (AKT) [5] is a strong Transformer-based baseline. Its goal is to predict whether a student will answer the next question correctly from the sequence of previous interactions. Since AKT is both effective and widely used in the literature, we chose to study it in detail and implement it from scratch.

For a student, we observe a sequence (q_t, c_t, r_t) , where q_t is the question, c_t its associated concept, and $r_t \in \{0, 1\}$ indicates correctness. Given the current pair (q_t, c_t) and the past interactions, the model predicts the probability of a correct response at time t .

At a high level, AKT can be summarized in three stages: *encoding*, *knowledge retrieval*, and *prediction*.

AKT Implementation Pipeline



Figure 2: Compact AKT pipeline

Encoding. AKT first builds embeddings for the question–concept pair and for the full interaction:

$$x_t = \mathbf{c}(c_t) + \mu(q_t)\mathbf{d}(c_t), \quad y_t = \mathbf{c}(c_t) + \mathbf{g}(r_t) + \mu(q_t)\mathbf{f}(c_t, r_t).$$

Here, $\mathbf{c}$ and $\mathbf{d}$ are concept embeddings, $\mathbf{g}$ is response embedding, $\mathbf{f}$ is concept–response embedding, while μ is a scalar question parameter. The vector x_t represents the current question, and y_t additionally includes the response. This factorized parameterization is more compact than learning a separate embedding for each question–concept pair. The resulting embeddings are then processed by self-attention blocks [6], yielding contextualized representations $\hat{x}_t$ and $\hat{y}_t$.

Knowledge retrieval. The model estimates the student’s knowledge state relative to the current question via an attention block where the query and key vectors are derived from the question representations $\hat{x}_t$, while the value vectors are taken from past interaction representations $\hat{y}_\tau$, with attention restricted to $\tau < t$ (the current interaction is not known yet).

Unlike standard attention, AKT introduces a *monotonic attention* mechanism to model forgetting. Attention scores are modulated by a distance-based decay:

$$s_{t,\tau} = \frac{\mathbf{Q}_t^\top \mathbf{K}_\tau}{\sqrt{m}} \exp(-\lambda d(t, \tau)),$$

where $\lambda > 0$ is a learned parameter. The distance $d(t, \tau)$ is not simply the temporal gap $t - \tau$, but is inferred from the attention distribution:

$$d(t, \tau) = \sqrt{(t - \tau) \sum_{i=\tau+1}^t \alpha'_{t,i}},$$

where $\alpha'_{t,i}$ are the attention weights before applying the decay. As a result, interactions that are both distant in time and weakly related to the current question are strongly downweighted, while relevant past interactions retain a significant influence.

The resulting representation is then passed through a feed-forward network to obtain the final knowledge state $h_t \in \mathbb{R}^D$.

Prediction. Finally, the model concatenates the retrieved knowledge state h_t with the current question embedding $\hat{x}_t$, then applies a feed-forward network followed by a sigmoid activation to predict the probability of a correct answer.

Implementation. We implemented AKT from scratch in PyTorch on the **ASSISTments 2009** dataset, a standard benchmark in knowledge tracing consisting of real student interaction logs from an online tutoring platform. Each interaction records a student answering a question associated with one or more knowledge components, together with a binary correctness label.

Our implementation follows the main architectural components of the original paper: question–concept embeddings, self-attention encoders for questions and past interactions, and a knowledge retriever based on monotonic attention. In particular, we explicitly coded the monotonic attention mechanism with learned decay, strict causal masking, residual connections, layer normalization, and feed-forward blocks.

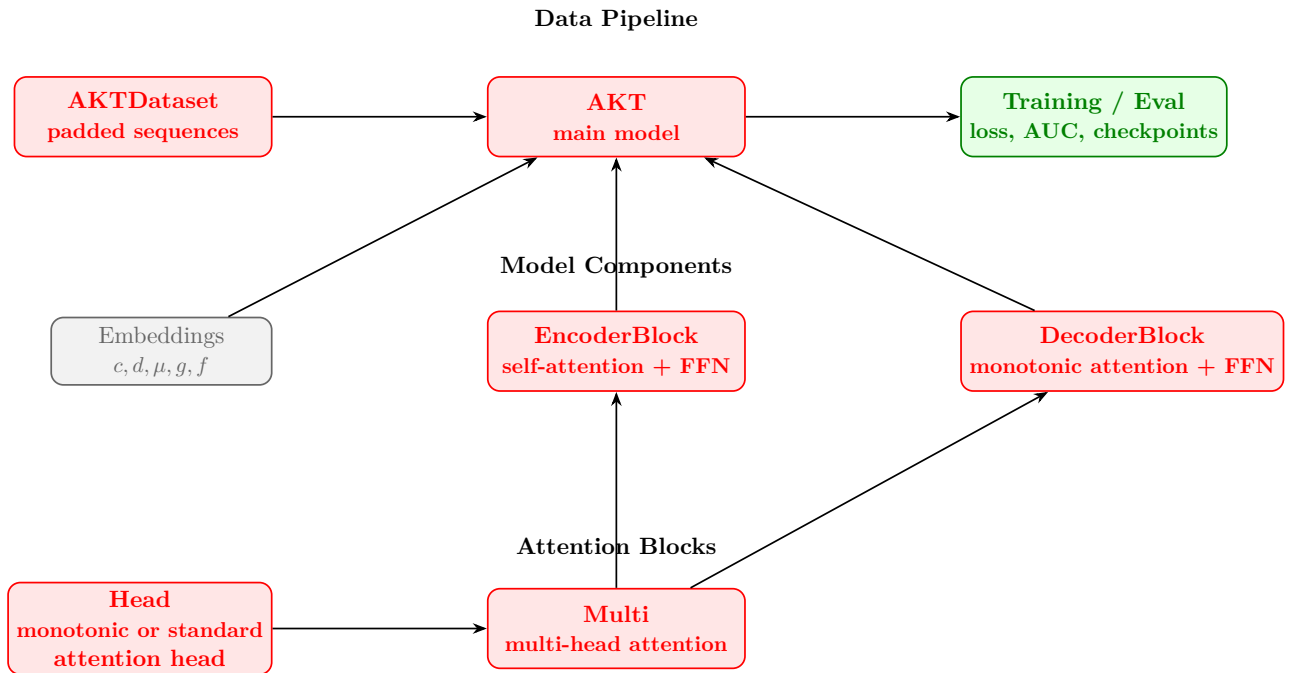


Figure 3: Structure of our AKT implementation. Classes are represented in red. **Head** implements a single attention head, optionally with monotonic decay; **Multi** combines several heads; **EncoderBlock** and **DecoderBlock** build the main Transformer-style components; and **AKT** assembles the full model for training and evaluation.

On the data side, we first sorted interactions by student and chronological order, mapped questions and skills to integer indices, and split the dataset by student into training (72%), validation (8%), and test sets (20%). Student trajectories were then divided into windows of length $T = 200$, with zero-padding and masking for shorter sequences and chunking for longer ones. We also implemented the full training and evaluation pipeline, including checkpointing, validation-loss monitoring, and final computation of accuracy and AUC on the test set. The model outputs predictions only for valid time steps, and for padded batches we compute the binary cross-entropy loss restricted to non-padded positions.

Compared with the original paper, we used a lighter configuration in order to keep the implementation computationally manageable.

Parameter	Ours	AKT paper
Embedding dimension D	32	128–256
Attention heads	8	8
Sequence length	200	200
Dropout	0.2	0.2
FFN hidden size	128	$\approx 4D$

This version has 236 512 parameters, much fewer than the original model, which largely explains the performance gap.

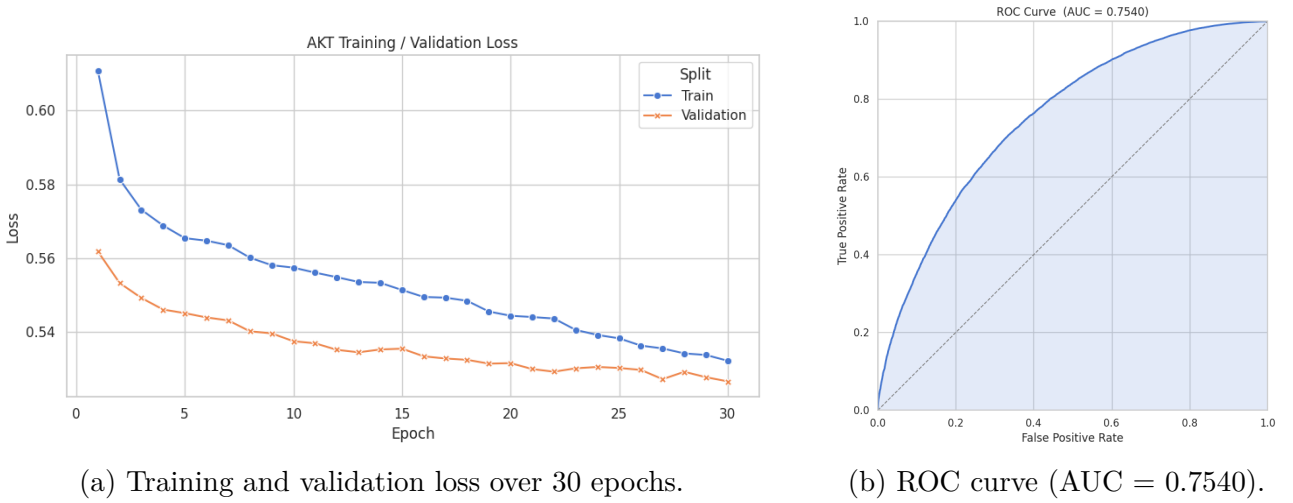


Figure 4: AKT training dynamics and test performance. The loss decreases steadily, indicating stable convergence, while the ROC curve shows good discriminative ability.

Results. Training was stable over 30 epochs on an NVIDIA RTX A5000 GPU (Figure 4a). On the test set, the model reached an accuracy of 0.7299 and an AUC of 0.7540. Although this remains below the 0.835 AUC reported in [5], the gap is consistent with our smaller embedding dimension and lighter tuning. Overall, our implementation reproduces the main ideas of AKT while remaining less computationally heavy.

This implementation gave us a concrete understanding of both the strengths and the limitations of modern deep KT models. On the one hand, AKT provides a flexible and expressive architecture capable of exploiting rich temporal dependencies. On the other hand, it remains relatively costly to train and is best suited to settings in which a large amount of historical data is available beforehand. This observation became particularly important when we turned to competitive programming, where interactions are shorter, more heterogeneous, and often arrive in an online fashion. These considerations motivated the second part of our work in knowledge tracing, namely the design of a live setting and the exploration of lighter models based on tabular foundation models.

1.2 LIVE KNOWLEDGE TRACING WITH TABULAR FOUNDATION MODELS

This section draws on our paper [7], published during our PSC at AIED 2026 (International Conference on AI in Education) under the title

Live Knowledge Tracing: Real-Time Adaptation with Tabular Foundation Models.

1.2.1 • MOTIVATION

Most knowledge tracing benchmarks come from educational platforms where students solve many short exercises on well-established topics. These datasets contain long interaction sequences and repeated exposure to the same items, enabling deep models such as deep KT and attentive KT to be trained offline on large historical logs.

Competitive programming presents a different setting. Problems are harder, sequences are shorter, and tasks are often new to all participants, especially during contests. Historical data on identical problems is typically unavailable, and interactions arrive progressively through submissions or live scoreboards. In such regimes, deep KT models may struggle, particularly in cold-start settings with few observations per student [8].

This motivated us to create *live knowledge tracing* (liveKT), where interactions arrive sequentially and predictions must be updated online. Retraining deep models whenever new data arrives is impractical in this setting.

Meanwhile, foundation models have shown strong generalization through *in-context learning*, adapting to new tasks directly from input examples without fine-tuning [9]. This paradigm has recently been extended beyond language to tabular [10] and graph data [11].

In this part of the PSC, we investigated the use of tabular foundation models for knowledge tracing without task-specific training.

We focus on a live setting where student interactions are observed over time, as in programming contests, and predictions must be updated in real time.

Our contributions are threefold: (i) we introduce a lightweight KT pipeline based on tabular foundation models and in-context learning, (ii) we formalize the liveKT setting, and (iii) we show empirically that this approach is significantly faster to deploy than standard deep KT models while remaining competitive in predictive performance.

1.2.2 • RELATED WORK

Tabular Foundation Models (TFMs) like TabPFN [12] and TabICL [10] are optimized for tabular structures. By leveraging in-context learning across both rows and features, TFMs can match the accuracy of extensively tuned gradient boosting models in mere seconds [13].

Despite this sample efficiency, TFMs remain under-explored for knowledge tracing, particularly in live environments where new interactions continuously emerge.

1.2.3 • TABULAR FOUNDATION MODELS FOR LIVE KNOWLEDGE TRACING

In the traditional knowledge tracing framework, we separate train and test students so that all interactions are available for train students while only the first $T - 1$ interactions are observed for test students. The model must predict the next outcome at time T . In what we call *live knowledge tracing* (liveKT), at time T , the model observes interactions for train students up to time T and for test students up to time $T - 1$. The model must again predict the next outcome for test students. This setting is harder than the traditional one (train students lose their final interactions as a training signal) but we argue it is more realistic, as it mirrors a classroom where all students complete tasks simultaneously over time.

Tabular foundation models (TFMs) operate by receiving a table of rows and columns and inferring, at test time, the values of a missing column for designated "test" rows; we refer to nanoTabPFN [14] for architectural details.

In our setup, each row in our dataset represents a single student interaction, parametrized by the student and the time of the interaction:

$$\mathcal{R}(u, t) = (u, q_u(t), s_u(t), c_u(t), t)$$

where u is the student ID, $t \in \{1, \dots, T\}$ is the interaction index, $q_u(t)$ is the ID of the question presented to student u at time t , $s_u(t)$ is the corresponding skill ID, and $c_u(t) \in \{0, 1\}$ indicates whether the student answered correctly. The index t is included explicitly since TFMs, unlike regular transformers, have no built-in positional encoding.

Students are split into I_{train} and I_{test} and we define the context set:

$$\begin{aligned} \mathcal{D}(T) = & \{\mathcal{R}(u, t), u \in I_{train}, t \in \{1, \dots, T\}\} \\ & \cup \{\mathcal{R}(u, t), u \in I_{test}, t \in \{1, \dots, T - 1\}\} \end{aligned}$$

The goal is to model for every test student $p(c_u(T) | \mathcal{D}(T))$.

To this end, we feed $\mathcal{D}(T)$ as the training rows and ask the model to predict the column $c_u(T)$ for $u \in I_{test}$. A key advantage of TFMs is that $\mathcal{D}(T)$ can be passed directly at inference time, whereas standard models such as DKT or AKT must first be trained on train-student data before producing predictions for test students.

1.2.4 • EXPERIMENTS

We evaluate the proposed approach on several datasets. Data is truncated to the first $T \in [5, 10, 15, 20]$ samples per user to simulate a live setting. We filter students having less than 5 interactions. We run a 5-fold cross-validation over students. Therefore, results are averaged

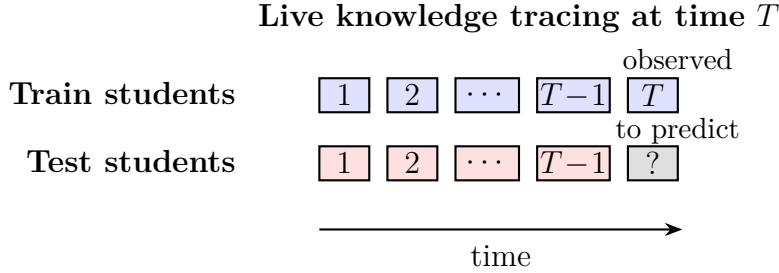


Figure 5: Live knowledge tracing setting. At time T , interactions are observed up to time T for train students, while for test students only interactions up to time $T - 1$ are available. The goal is to predict the hidden response at time T .

across 5 runs, and data from 20% of users is used for evaluation. We report the area under the ROC curve (AUC) and wall-clock latency. Experiments were run on one NVIDIA RTX 4000 GPU with 20 GB of RAM during at most 115 epochs. We release our code¹.

Datasets. We use three datasets (Table 1): ASSISTments2009 [15], a mathematics tutoring dataset from Worcester Polytechnic Institute; and POJ and Codeforces², two programming competition datasets from Peking University Online Judge and the Codeforces platform. Dataset statistics are reported on the raw data; interactions are truncated to at most $T_{\max} = 20$ per student in all experiments.

Table 1: Raw dataset statistics (before truncation to $T_{\max} = 20$ interactions per student).

	ASSISTments 2009	POJ	Codeforces
Domain	Mathematics	Programming	Programming
# Students	4,217	433	500
# Questions	26,688	2,622	5,857
# Skills	123	85	36
# Interactions	346,860	796,249	754,772

Models. **DKT** [16] and **AKT** [5] are implemented via pyKT [17], using the best hyperparameters reported in previous experiments. **LR** is logistic regression on categorical features. **GBM** is a histogram-based gradient boosting model from skrub³ and scikit-learn [18]. **TabPFN** (v2.6) and **TabICL** (v2) are tabular foundation models; the same instance is used across all datasets as they do not require hyperparameter tuning. TabPFN has complexity $O(N^2T + NT^2)$ while TabICL has complexity $O(N^2 + NT^2)$ where N is the number of users and T is the sequence length.

¹<https://anonymous.4open.science/r/liveKT-2512/README.md>

²<https://github.com/JonathanSilver/pyKT/tree/main/data>

³<https://skrub-data.org/>

1.2.5 • RESULTS

Results are shown in Table 2. We reported all models better than logistic regression in Figure 6. Time refers to the median time per T , averaged across $T = 5, \dots, 20$, including both training and testing across all epochs. TabPFN and TabICL do not perform training but look at training data once, at test time. Across datasets, TFMs achieve competitive AUC compared to fully trained deep models while dramatically reducing time. In particular, on POJ and Codeforces, TabPFN and TabICL systematically outperform AKT and DKT. GBM proves a surprisingly hard baseline, with a few cases where it outperforms both tabular (TabICL, TabPFN) and state-of-the-art deep trained models (AKT, DKT). On ASSISTments 2009, AKT takes 436s to converge after 105 epochs, while TabICL finishes in 7.7s, a $57\times$ speedup. TabICL achieves an average speedup of $53\times$ over AKT. We note that on POJ, DKT has very low AUC for $T = 5$, consistent with prior work showing DKT may overfit in cold-start settings [8]; early stopping between epochs 11 and 15 confirms this tendency.

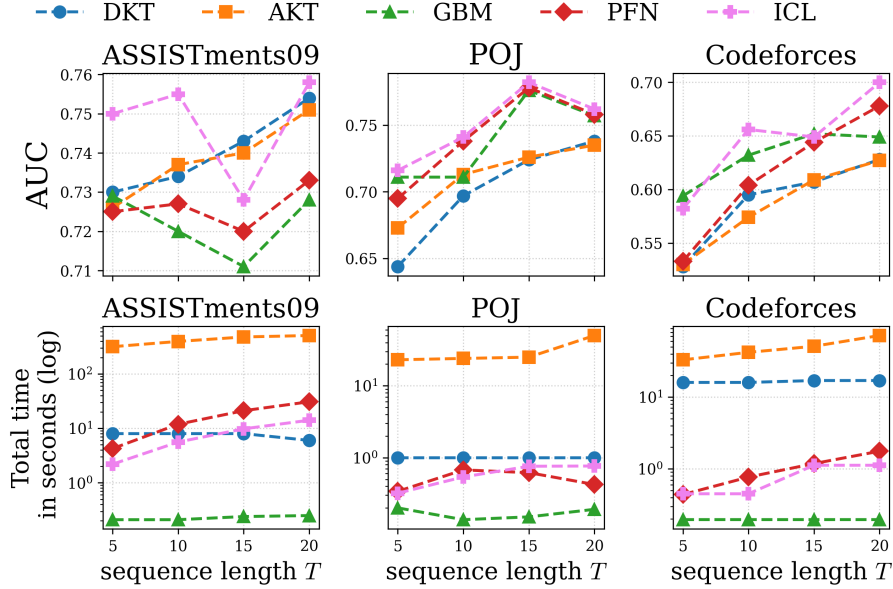


Figure 6: Results for ASSISTments 2009 and POJ. Top: AUC, bottom: total time.

1.2.6 • DISCUSSION AND CONCLUSION

We presented a tabular foundation model (TFM) approach to knowledge tracing that removes the need for offline training and is suitable for live environments such as programming competitions. Our experiments show that TFMs achieve competitive AUC while being orders of magnitude faster than the other baselines, performing consistently well even on small datasets. This indicates that tabular foundation models may be particularly suited for knowledge tracing on smaller datasets in a classroom, compared to other deep learning-based knowledge tracing models.

The key insight is that TFMs learn to align testing and training samples via an attention

Table 2: Performance (AUC) of models for different sequence lengths T . Bold indicates the best model for that sequence length, underline indicates the runner-up.

Data	Model	Time	Epochs	Sequence length T			
				5	10	15	20
ASSISTments	LR	0.04s	1	0.541	0.519	0.567	0.559
	DKT	8s	19	<u>0.730</u>	0.734	0.743	<u>0.754</u>
	AKT	436s	105	0.726	<u>0.737</u>	<u>0.740</u>	0.751
	GBM	0.22s	1	0.729	0.720	0.711	0.728
	TabPFN	16.5s	0	0.725	0.727	0.720	0.733
	TabICL	7.7s	0	0.750	0.755	0.728	0.758
POJ	LR	0.01s	1	0.491	0.506	0.502	0.491
	DKT	1s	21	0.644	0.697	0.724	0.738
	AKT	25s	53	0.673	0.713	0.726	0.735
	GBM	0.17s	1	<u>0.711</u>	0.711	0.776	0.757
	TabPFN	0.52s	0	0.695	<u>0.738</u>	<u>0.778</u>	<u>0.758</u>
	TabICL	0.65s	0	0.716	0.741	0.782	0.762
Codeforces	LR	0.01s	1	0.523	0.572	0.489	0.491
	DKT	1s	16	0.528	0.595	0.607	0.628
	AKT	24s	47	0.530	0.574	0.609	0.627
	GBM	0.2s	1	0.594	<u>0.632</u>	0.652	0.649
	TabPFN	1s	0	0.533	0.604	0.644	<u>0.678</u>
	TabICL	0.8s	0	<u>0.582</u>	0.656	<u>0.649</u>	0.700

mechanism. While AKT attends only along the sequence of time steps for a single student, TFMs attend across all samples, previous time steps for the same student and other students, enabling them to identify the most relevant training sequences for each prediction. This also provides a degree of transparency: by analyzing attention weights, one could identify “nearest neighbor” students whose behaviors most closely align with the current test student.

One limitation of our approach is that its performance depends on the specific pretrained weights of the underlying foundation model. Even so, this study highlights the potential of in-context learning for knowledge tracing in live environments, especially when fast adaptation and low deployment cost are more important than extensive task-specific training. More broadly, in the context of our PSC, this work shows how our initial study of standard knowledge tracing models gradually evolved into a more original contribution, driven by the practical constraints of competitive programming and by our desire to design methods better suited to this setting.

2

DYNAMIC COGNITIVE DIAGNOSIS

While the first part of this report focused on predicting future student performance from interaction data, an intelligent tutoring system must also be able to infer a learner's current global knowledge state in order to recommend suitable exercises. This raises a different but closely related problem: rather than predicting the next answer, we now seek to determine which knowledge components are already mastered and which are not.

Our work on this part began from a simple question: how can we assess a student's mastered knowledge components efficiently? A naive strategy would consist in querying the learner on every single knowledge component, which quickly becomes impractical in large domains. The key idea is instead to exploit prerequisite relations between concepts. For example, if a student masters Dijkstra's algorithm, one may reasonably infer that they also understand more elementary notions such as graphs, edges, and vertices. Our goal in this part is therefore to use such dependencies to reconstruct a learner's hidden knowledge state using as few queries as possible.

This perspective naturally led us to a mathematical formulation in which the domain is represented as a prerequisite structure and a learner's knowledge state is modeled as a subset consistent with this structure. We first introduce this formalization. We then present a general theoretical framework, based on order ideals and central elements, and derive an algorithm that solves the querying problem with optimal complexity by combining several theoretical results from the literature. Finally, we discuss how this framework can be implemented in practice through an MCMC-based approach, which, to the best of our knowledge, is new in this context.

This model is, of course, a simplification. In practice, mastery is rarely strictly binary, and a correct answer to a query does not guarantee complete understanding of the corresponding concept.

It is also worth noting that the problem of inferring a learner's knowledge via prerequisite structures has been studied within the Knowledge Space Theory: [19]. Traditional KST models are typically stochastic; they assume an underlying probability distribution over the possible knowledge states and incorporate parameters to update these probabilities. In contrast, our approach does not suppose any prior probabilistic distribution regarding the learner's knowledge states.

However, we chose to explore this model for the richness of the theoretical bounds and algorithmic results it makes possible.

2.1 MATHEMATICAL FORMULATION

To address the question of efficient knowledge-state querying, our first step was to provide a mathematical formalization of it. We represent the domain knowledge as a partially ordered set (poset). We define a relation $\preceq$ such that $x \preceq y$ if the Knowledge Component x is a prerequisite for the Knowledge Component y . For example, basic “addition” is a prerequisite for understanding “polynomials,” so addition $\preceq$ polynomials. However, not all KCs are comparable; for instance, knowing the “Euclidean algorithm” is neither a prerequisite for nor dependent on “Thales’s theorem.”

Definition 2.1 (Knowledge Poset) *Let $P = (V, \preceq)$ be a finite poset where V is the set of elements (practically KCs) and the partial order $\preceq$ represents the prerequisite relationships. For $x, y \in V$, $x \preceq y$ means that x is a prerequisite of y .*

From a practical and implementation point of view, imagining posets as Directed Acyclic Graphs (DAGs) is highly beneficial.

Proposition 2.1 (Equivalence between Posets and DAGs) *Any finite partially ordered set $P = (V, \preceq)$ can be equivalently represented as a Directed Acyclic Graph (DAG), and conversely, the transitive closure of any DAG forms a valid poset.*

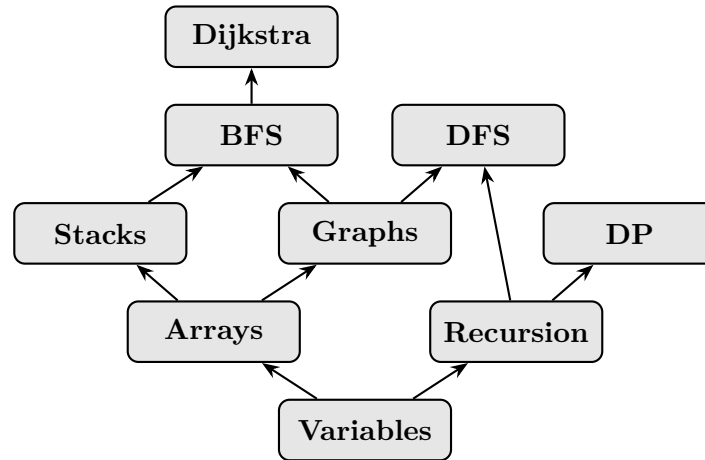


Figure 7: A prerequisite poset modeling a graph algorithms prerequisite example. Arrows indicate that the lower node must be mastered before the upper node ($x \preceq y$).

A set of KCs is naturally modeled as a DAG, where directed edges correspond to the immediate prerequisite dependencies. Taking the transitive closure of this DAG yields our poset P . In the example, even if there is no direct edge from ‘Variables’ to ‘BFS’, the former is a prerequisite of the latter by transitivity.

From here onward, we assume that a student’s knowledge state verifies: if a student masters KC y , they must also master all its prerequisites x . We formalize this through a consistent binary labeling.

Definition 2.2 (Consistent Labeling) A knowledge state is a binary function $f : V \rightarrow \{0, 1\}$. The consistency condition dictates that if an element has a label of 1, all its prerequisites must also have a label of 1. Formally, for all $x, y \in V$, if $x \preceq y$ and $f(y) = 1$, then $f(x) = 1$.

In the mathematical theory of posets, a subset that satisfies this downward-closed property is known as an order ideal.

Definition 2.3 (Order Ideal) An order ideal of a poset $P = (V, \preceq)$ is a subset $I \subseteq V$ such that if $y \in I$ and $x \preceq y$, then $x \in I$.

From these definitions, it follows that the subset of elements labeled 1, denoted $I = \{v \in V \mid f(v) = 1\}$, forms an order ideal in P .

The problem of assessing a student's knowledge state thus reformulates into the following: we want to identify a hidden order ideal I . At each step, we can query an element $x \in V$ to know whether it belongs to the ideal I or not. Our goal is to entirely determine this unknown order ideal using the minimum number of queries. Let $\mathcal{I}(P)$ denote the set of all valid order ideals of P (i.e. all valid possible knowledge states).

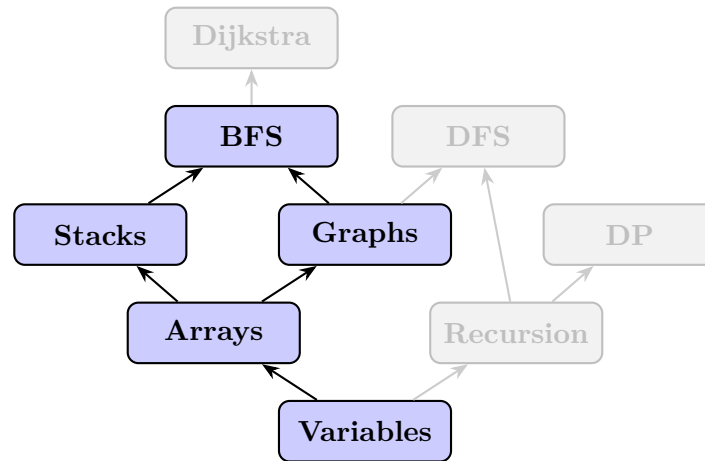


Figure 8: A valid knowledge state (order ideal) mapped onto the prerequisite poset. The mastered concepts (Variables, Arrays, Stacks, Graphs, BFS) are highlighted in blue. Notice that this state satisfies the consistency condition: it is perfectly downward-closed, meaning every prerequisite of a mastered concept is also mastered.

2.2 FIRST APPROACH: CHAIN DECOMPOSITION AND BINARY SEARCH

Our first approach is motivated by the simple case in which the prerequisite structure forms a single chain. If the poset is a completely ordered set, the consistency condition implies that there is a specific boundary where the chain of mastered elements stops. Finding this

boundary simply requires searching for the transition point between elements labeled 1 and elements labeled 0. This can be efficiently done in $O(\log n)$ queries using a standard binary search, where $n = |V|$ is the total number of elements.

This simple case naturally motivates an algorithm for the general case: if we can decompose our prerequisite graph into several independent chains, we can simply apply our binary search strategy independently to each chain. This concept is visualized, which illustrates how a Directed Acyclic Graph (DAG) representing prerequisites can be partitioned into disjoint linear chains.

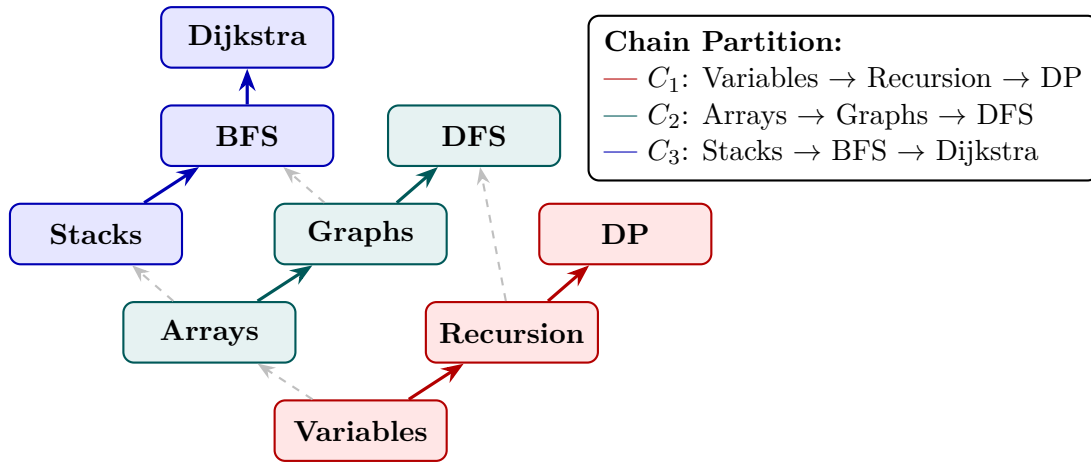


Figure 9: A valid chain decomposition of the curriculum poset into 3 disjoint chains (C_1 , C_2 , C_3). Solid colored lines represent the sequence within each chain, while dashed gray lines indicate structural prerequisites that cross between chains.

To formalize this decomposition, we rely on a fundamental result in order theory. First, let us define an **antichain** as a subset of elements where no two elements are comparable (representing completely independent elements), and a **chain** as a totally ordered subset of elements.

Theorem 2.1 (Dilworth's Theorem) *In any finite partially ordered set $P = (V, \preceq)$, the maximum size of an antichain is equal to the minimum number of chains needed to partition the elements of V .*

As a heuristic, we want to minimize the number of binary searches we do, hence we want to divide the poset into the minimum possible number of chains. Dilworth's Theorem guarantees that this minimum number is the width w of the poset (the size of its maximum antichain).

• ALGORITHM DESCRIPTION

The procedure can be summarized by the following algorithmic steps:

1. **Decomposition:** Find a "suitable" chain decomposition. This partitions the set of elements V into w disjoint chains, denoted as $C_1, C_2, \dots, C_w$.
2. **Independent Searching:** For each chain C_i (for i from 1 to w):

- Treat C_i as a totally ordered array.
 - Perform a binary search on C_i to find the highest element in the chain that evaluates to 1. Because of the prerequisite constraints, every element below it in the chain will also necessarily be 1, and every element above it will be 0.
3. **Aggregation:** The union of the downward-closed prefixes found in all w chains constitutes the entirety of the hidden ideal I .

2.3 SECOND APPROACH: THEORETICAL BOUND VIA THE CENTRAL ELEMENT THEOREM

While the previous approach provides useful intuition, it does not yield a general optimality guarantee. We can reframe the problem from a different perspective. At the beginning of the assessment, the set of all possible knowledge states is exactly the set of all valid order ideals, $\mathcal{I}(P)$. Among all these ideals, exactly one represents the true knowledge state of the student.

Every time we query an element $x \in V$, the binary response (mastered or not mastered) partitions our search space into two sets: the ideals containing x and the ideals not containing x . Based on the answer, we eliminate one of these sets. To guarantee fast convergence to the unique correct ideal, we must ensure that in the worst-case scenario, the number of eliminated ideals is as large as possible. This approach is intrinsically identical to a binary search, but generalized to the abstract space of order ideals.

However, explicitly generating and tracking the set of remaining ideals in memory is computationally hard.

Proposition 2.2 (Bounds on the Number of Ideals) *For any finite poset $P = (V, \preceq)$ with $n = |V|$ elements, the number of valid order ideals $|\mathcal{I}(P)|$ is bounded between:*

$$n + 1 \leq |\mathcal{I}(P)| \leq 2^n$$

Using the concepts introduced in our first approach, these bounds map to the two extreme structural cases of a poset:

- The lower bound $(n + 1)$ is achieved when P is a **chain**. Here, every element is a prerequisite for the next, and the only valid ideals are the $n + 1$ prefixes of this sequence.
- The upper bound (2^n) is achieved when P is an **antichain**. Here, no prerequisites exist between any elements, meaning every possible subset of V forms a valid ideal.

Because the search space $\mathcal{I}(P)$ can be exponentially large, keeping an explicit track of the remaining valid ideals is intractable. Fortunately, we do not need to explicitly maintain this set. When we eliminate a subset of ideals, the remaining valid ideals naturally map to the ideal space of a smaller modified poset that is easy to construct.

Lemma 2.1 (Poset Reduction and Bijection) *Let $P = (V, \preceq)$ be the current poset and let $x \in V$ be the queried element.*

- **Case 1:** *If x has a label of 1, let $A_x = \{y \in V \mid y \preceq x\}$ be the set of its ancestors (including x). There exists a bijection between the ideals of P containing x and the ideals of a smaller poset $P_{x=1}$, obtained by removing A_x from V .*
- **Case 2:** *If x has a label of 0, let $D_x = \{y \in V \mid x \preceq y\}$ be the set of its descendants (including x). There exists a bijection between the ideals of P not containing x and the ideals of a smaller poset $P_{x=0}$, obtained by removing D_x from V .*

Proof. For **Case 1**, let $\mathcal{I}_1 = \{I \in \mathcal{I}(P) \mid x \in I\}$. Because any ideal is downward-closed, $x \in I \implies A_x \subseteq I$ for all $I \in \mathcal{I}_1$. We define the reduced poset $P_{x=1} = (V \setminus A_x, \preceq)$ where the partial order is simply restricted to the remaining elements. We define the mapping $f : \mathcal{I}_1 \rightarrow \mathcal{I}(P_{x=1})$ as $f(I) = I \setminus A_x$.

- **Well-defined:** If $y \in I \setminus A_x$ and $z \preceq y$ with $z \in V \setminus A_x$, then $z \in I$ because I is an ideal in P . Thus, $z \in I \setminus A_x$, so $f(I)$ is indeed an ideal in $P_{x=1}$.
- **Injectivity:** If $I \setminus A_x = J \setminus A_x$, then $(I \setminus A_x) \cup A_x = (J \setminus A_x) \cup A_x$, which implies $I = J$ (since both I and J contain A_x).
- **Surjectivity:** Let $K \in \mathcal{I}(P_{x=1})$. Define $I = K \cup A_x$. We must show $I \in \mathcal{I}(P)$. Let $y \in I$ and $z \preceq y$. If $y \in A_x$, then $z \preceq y \preceq x$, so $z \in A_x \subseteq I$. If $y \in K$, then either $z \in A_x \subseteq I$, or $z \in V \setminus A_x$. In the latter case, because K is an ideal in $P_{x=1}$, $z \in K \subseteq I$. Thus, I is an ideal in P , it contains x , and $f(I) = K$. This establishes the bijection.

Case 2 can be done in a similar way. ■

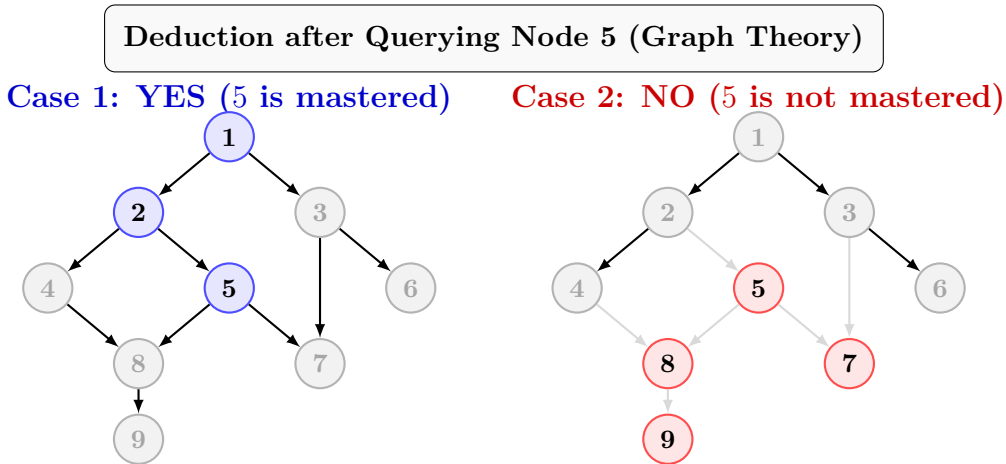


Figure 10: Side-by-side comparison of the knowledge state deduction based on the response to Node 5. Mastered KCs are blue, Not Mastered KCs are red, and the rest whose state is not yet determined are gray. In Case 1 (YES), mastery of Node 5 allows immediate deduction that prerequisites 1 and 2 are mastered. In Case 2 (NO), Node 5 is not mastered, meaning descendants 7, 8, and 9 cannot be mastered.

Because of this lemma, we only need to continually update the poset graph itself, rather than the list of ideals. The core algorithmic challenge thus reduces to a single question: *What is the best element $x \in V$ to query?*

To simulate a balanced binary search, the optimal element would be one that belongs to exactly half of the current valid ideals. Querying it would cut the search space by 50% regardless of the student's answer. To guarantee that such an element (or a close approximation) always exists, we rely on a fundamental result by Linial and Saks [20].

Theorem 2.2 (Central Element Theorem) *In any finite partially ordered set $P = (V, \preceq)$, there always exists a central element $x \in V$, i.e., such that the proportion of ideals in $\mathcal{I}(P)$ containing x lies strictly between bounds:*

$$\delta \leq \frac{|\{I \in \mathcal{I}(P) \mid x \in I\}|}{|\mathcal{I}(P)|} \leq 1 - \delta$$

where $\delta = \frac{3 - \log_2(5)}{4} \approx 0.17$ is a constant.

It is worth noting that the theoretical limit is tightly bounded. As mentioned in the same paper [20], a personal communication with J. Shearer established that there exist posets containing no δ -central element for any $\delta > 0.197$.

• ALGORITHM DESCRIPTION

The search strategy proceeds recursively:

1. Given the current poset P , identify a central element x satisfying the Linial-Saks bounds.
2. Query the user to determine if x is mastered.
3. Apply Lemma 2.1:
 - If answered 1: Replace P with $P_{x=1}$ (remove x and all its ancestors).
 - If answered 0: Replace P with $P_{x=0}$ (remove x and all its descendants).
4. Repeat the process until V is empty, at which point the complete initial ideal I is reconstructed.

• COMPLEXITY ANALYSIS

We establish both the lower and upper bounds on the number of queries to prove the optimality of this algorithm.

Lower Bound: From an information-theoretic point of view, any algorithm must distinguish between the $|\mathcal{I}(P)|$ possible initial knowledge states. Since each query provides a binary response (1 bit of information), the decision tree modeling the algorithm's execution must have at least $|\mathcal{I}(P)|$ leaves. The depth of such a binary tree is at least $\lceil \log_2 |\mathcal{I}(P)| \rceil$. Thus, any algorithm requires a minimum of $\Omega(\log |\mathcal{I}(P)|)$ queries.

Upper Bound: Let S_k denote the number of valid ideals remaining in the search space after k queries. Initially, $S_0 = |\mathcal{I}(P)|$. By the Central Element Theorem, querying the central element ensures that the size of the search space is multiplied by at most $(1 - \delta)$ at each step, regardless of the user's answer. Therefore, after Q queries, the number of remaining ideals is bounded by:

$$S_Q \leq |\mathcal{I}(P)| \cdot (1 - \delta)^Q$$

The algorithm terminates when the search space is reduced to a single ideal. So the algorithm asks a maximum number of question of

$$Q = \left\lceil \frac{\log |\mathcal{I}(P)|}{\log(1/(1 - \delta))} \right\rceil$$

yielding an upper bound of $O(\log |\mathcal{I}(P)|)$.

Conclusion:

Hence for the optimal querying problem, the number of questions to ask is in $\Theta(\log |\mathcal{I}(P)|)$, and the algorithm provided achieves this complexity.

2.4 PRACTICAL IMPLEMENTATION: FINDING THE CENTRAL ELEMENT

In the algorithm description above, we glossed over a critical practical detail: how to efficiently find a central element.

This problem turns out to be hard for general posets. Identifying a central element requires determining the proportion of ideals that contain a given element, which in turn requires counting the total number of ideals in the poset. However, as established by Provan and Ball [21], exact counting of ideals in a general poset is a #P-complete problem.

Fortunately, it is not strictly necessary to find the absolute mathematically optimal central element. Our algorithm only requires an element whose inclusion probability is roughly close to 50%. If we can approximate the proportion of ideals containing each element $v \in V$, denoted as $p(v)$, we can simply select the element v that minimizes $|p(v) - 0.5|$.

To approximate these probabilities without explicitly enumerating the exponentially large state space $\mathcal{I}(P)$, we design a randomized algorithm using Markov Chain Monte Carlo methods. The core idea is to simulate a random walk over the space of all valid ideals. If we design this walk such that its stationary distribution is uniform over $\mathcal{I}(P)$, we can estimate $p(v)$ by simply observing how frequently v appears in the generated ideals.

MCMC Design We define a Markov Chain where the state space Ω is the set of all valid order ideals $\mathcal{I}(P)$. The transition from a current ideal I_t to the next ideal I_{t+1} operates as follows:

1. Choose an element $v \in V$ uniformly at random.
2. If $v \in I_t$, we propose removing it ($I' = I_t \setminus \{v\}$). If $v \notin I_t$, we propose adding it ($I' = I_t \cup \{v\}$).
3. Check if the candidate I' is a valid order ideal.
 - If I' is an ideal, the transition is accepted: $I_{t+1} = I'$.
 - otherwise: $I_{t+1} = I_t$.

Convergence to the Uniform Distribution We need the Markov Chain to converge to the uniform distribution $\pi(I) = \frac{1}{|\mathcal{I}(P)|}$. This is guaranteed by the Fundamental Theorem of Markov Chains, provided the chain satisfies three properties:

- **Symmetry:** Consider any two adjacent valid ideals I and I' that differ by exactly one element v (without loss of generality, assume $I' = I \cup \{v\}$). To transition from $I \rightarrow I'$, the algorithm must select the specific element v from the set of all elements V , which happens with probability $\frac{1}{|V|}$. Since I' is valid, the proposed addition is always accepted. Conversely, to transition from $I' \rightarrow I$, the algorithm must select the same element v , again with probability $\frac{1}{|V|}$, and since I is valid, the proposed removal is always accepted. Thus, the transition probabilities are symmetric: $P(I, I') = P(I', I) = \frac{1}{|V|}$.
- **Aperiodicity:** Because transitions can be rejected (e.g., if the algorithm proposes adding an element without its prerequisites), the chain has a positive probability of remaining in the same state ($I_{t+1} = I_t$). Hence the chain is aperiodic.
- **Irreducibility:** From any valid ideal I , we can repeatedly select maximal elements within I and remove them. This path guarantees we can always reach the empty set $\emptyset$. Because our transition matrix is symmetric (and thus reversible), any path can be traversed in reverse, meaning we can reach any state J from any state I via $\emptyset$.

Because the chain is irreducible and aperiodic, the Fundamental Theorem of Markov Chains guarantees that a strictly unique stationary distribution π exists. We can prove that this unique distribution is uniform.

Let $N = |\mathcal{I}(P)|$ be the total number of valid ideals. We define the uniform distribution $\pi(I) = \frac{1}{N}$ for all $I \in \mathcal{I}(P)$. A distribution is stationary if it satisfies for every state J : $\sum_I \pi(I)P(I, J) = \pi(J)$

By substituting $\pi(I) = \frac{1}{N}$ into the left side of the equation, we obtain (using symmetry):

$$\sum_I \frac{1}{N} P(I, J) = \frac{1}{N} \sum_I P(J, I) = \frac{1}{N} \times 1 = \frac{1}{N} = \pi(J)$$

This proves that the uniform distribution satisfies the stationary condition. Combined with the uniqueness guaranteed by irreducibility and aperiodicity, the stationary distribution is uniform over all valid ideals.

Estimation via the Ergodic Theorem Because the Markov chain is irreducible and aperiodic, and because its stationary distribution is uniform over $\mathcal{I}(P)$, the Ergodic Theorem applies. For any function $f : \mathcal{I}(P) \rightarrow \mathbb{R}$, we have $\frac{1}{T} \sum_{t=1}^T f(I_t) \xrightarrow[T \rightarrow \infty]{a.s.} \mathbb{E}_{I \sim \pi}[f(I)]$.

To estimate the probability that a given element $v \in V$ belongs to a uniformly random ideal, we choose the indicator function

$$f_v(I) = \mathbf{1}_{\{v \in I\}}.$$

In that case, $\mathbb{E}_{I \sim \pi}[f_v(I)] = \frac{|\{I \in \mathcal{I}(P) | v \in I\}|}{|\mathcal{I}(P)|} = p(v)$. Therefore,

$$\hat{p}(v) := \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{v \in I_t\}} = \frac{1}{T} \sum_{t=1}^T f_v(I_t) \xrightarrow[T \rightarrow \infty]{a.s.} p(v).$$

By computing $\hat{p}(v)$ for all $v \in V$, we can approximate a central element by selecting

$$v^* \in \arg \min_{v \in V} \left| \hat{p}(v) - \frac{1}{2} \right|.$$

Practical implementation. In practice, we implement the Markov chain by maintaining the current ideal $A \subseteq V$ together with its complement $B = V \setminus A$. Recall that a move is valid only if the resulting set remains an order ideal. Rather than checking this condition from scratch at every step, we keep track of two auxiliary quantities:

- for each $v \in B$, its number of predecessors still lying in B , denoted `in_degree_B[v]`;
- for each $v \in A$, its number of successors still lying in A , denoted `out_degree_A[v]`.

An element $v \in B$ can be added to A if and only if `in_degree_B[v] = 0`, and an element $v \in A$ can be removed from A if and only if `out_degree_A[v] = 0`. This allows each transition to be implemented in time proportional only to the local degree of the chosen vertex.

To estimate the inclusion probabilities $p(v)$, whenever a vertex enters A , we store the current time step, and whenever it leaves A , we add the duration of its stay to a running counter. Dividing by the total number of iterations then gives an empirical estimate of $p(v)$.

The practical procedure is then as follows:

1. Initialize the current ideal as $A = \emptyset$, and let $B = V$.
2. For each vertex $v \in V$, initialize:
 - `in_degree_B[v]` (resp. `out_degree_A[v]`) as the number of predecessors (resp. successors) of v in B (resp. A)
3. Repeat the following procedure for a large number N of iterations:
 - (a) Choose a vertex $k \in V$ uniformly at random.
 - (b) If $k \in B$ and `in_degree_B[k] = 0`, then add k to A .
 - Record the time at which k entered A .

- Update the counters of the neighbors of k to reflect that k is no longer in B and is now in A .
- (c) If $k \in A$ and $\text{out_degree_A}[k] = 0$, then remove k from A .
- Add the time spent by k in A to its cumulative occupation counter.
 - Update the counters of the neighbors of k to reflect that k is no longer in A and is now in B .
- (d) If neither condition is satisfied, leave the state unchanged.
4. After the final iteration, add the remaining occupation time of every vertex still in A .
 5. Normalize the cumulative occupation times by N in order to obtain empirical estimates $\hat{p}(v)$ of the inclusion probabilities.

Complexity. Let $n = |V|$ and $m = |E|$. Initializing the current ideal and the auxiliary counters takes $O(n + m)$ time. Each iteration of the Markov chain only updates the local neighborhood of the selected vertex, so its cost is proportional to the in-degree and out-degree of that vertex. Hence, if the walk is run for N iterations, the total time complexity is $O(n + m + N\Delta)$ in the worst case, where Δ denotes the maximum total degree of a vertex. In sparse graphs, this is typically close to linear in N . The memory complexity is $O(n + m)$.

3

KNOWLEDGE MAPPING AND PRACTICAL APPLICATIONS

The previous part introduced a theoretical framework for inferring a learner's knowledge state from a prerequisite structure. To make such ideas usable in practice, we needed a concrete representation of the competitive programming domain and tools capable of exploiting it. This third part therefore turns to practical applications. We first study how competitive programming can be represented, through geometric visualizations of tags and prerequisite graphs constructed from data and large language models. We then present *CFTracker*, a web application designed to analyze a user's profile and recommend suitable problems.

3.1 DOMAIN REPRESENTATION

A prerequisite-based tutoring system first requires a representation of the domain itself. In competitive programming, this means identifying relevant skills, understanding how they relate

to one another, and organizing them into a structure that can support diagnosis and recommendation. We explored this question through two complementary directions: geometric visualization of the domain and explicit construction of dependency graphs.

3.1.1 • COMPETITIVE PROGRAMMING DOMAIN VISUALIZATION

Some competitive programming websites, including Codeforces and USACO Guide, already propose pre-made tags to their problemset, with around 5 tags on average per problem. Therefore, we used the tags prepared by these websites in order to create some visualisations of the competitive programming domain.

To do so, we needed a notion of semantic proximity between tags. We used a sentence encoder, Ollama’s MiniLM, which maps short text descriptions to vectors in a high-dimensional space of dimension 384. In order to provide enough context to the encoder, we enriched each tag with a short explanatory sentence, for example: *Greedy: build the answer by repeatedly taking a locally optimal valid choice*. Once embedded, these vectors were projected into two dimensions using PCA, t-SNE, and UMAP using the scikit-learn library in Python and obtained the visualizations in figure 14.

PCA looks for a k -dimensional space in which the projected points are as spread out as possible, t-SNE aims at preserving local neighborhood structures by matching probability distributions between the high-dimensional and low-dimensional spaces, and UMAP assumes that the data lies on a Riemannian manifold and seeks to preserve both local and global structure.

These visualizations provide an interpretable map of the domain. Close concepts such as *Tree* and *Minimum Spanning Tree* remain near one another across all three methods, while more distinct areas such as *Divisibility* and *Game Theory* are more clearly separated by t-SNE and UMAP than by PCA. This suggests that nonlinear methods are better suited to preserving the semantic structure of the domain.

It is also worth noting that these techniques could be extended to three-dimensional embeddings, which would likely make some clusters and relationships even easier to visualize.

3.1.2 • DEPENDENCY GRAPH GENERATION

Beyond geometric visualization, we also needed a representation that could directly serve as input for the methods developed in Part 2. More precisely, our goal was to construct prerequisite graphs for the domain of competitive programming, so that the theoretical framework based on order ideals and adaptive querying could be applied in practice.

To this end, we built directed graphs whose vertices represent competitive programming skills and whose edges encode possible prerequisite relations. Since a single problem may involve several skills, we allow each problem to be associated with multiple knowledge components, each corresponding to a distinct skill. To define these components, we used the tags provided with competitive programming problems, since they offer a natural and interpretable description of the skills involved in solving them. More specifically, we relied on the tags from the TACO

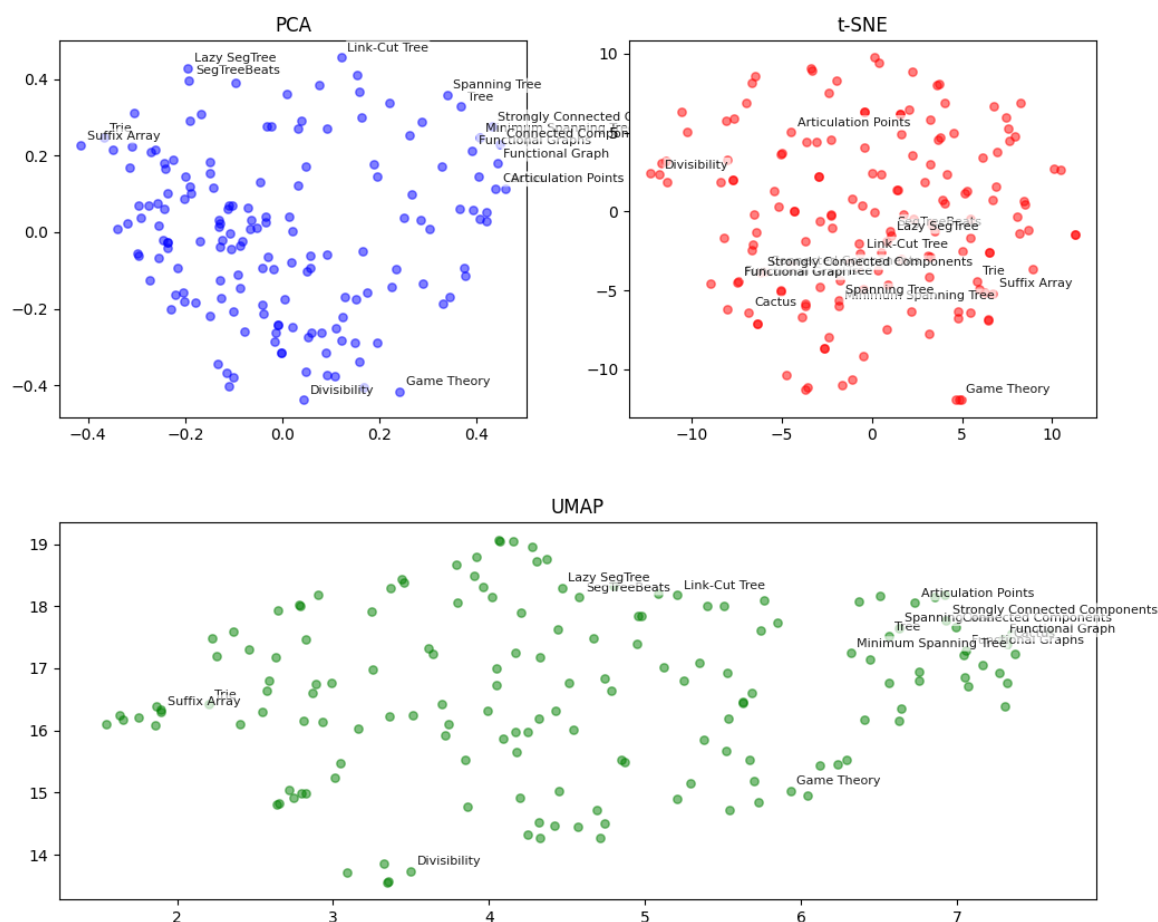


Figure 11: Dimensionality reduction algorithms on embedded sentences linked to 162 competitive programming tags extracted from the Codeforces and USACO Guide websites.

dataset [22]. TACO (*Topics in Algorithmic COde generation*) aggregates problems from several competitive programming platforms and maps a large set of original tags to 36 higher-level categories.

A first way to construct such a graph from this dataset is to compute conditional probabilities between tags. Given two tags A and B , we consider the probability that a problem labeled with B is also labeled with A . If problems involving B almost always also involve A , this suggests that A may act as a prerequisite for B . We orient edges according to this asymmetry, then remove weaker relations by thresholding. Since this procedure does not automatically guarantee acyclicity, the threshold can be increased until the resulting graph becomes acyclic. Applied to TACO, this yields the graph shown in Figure 12.

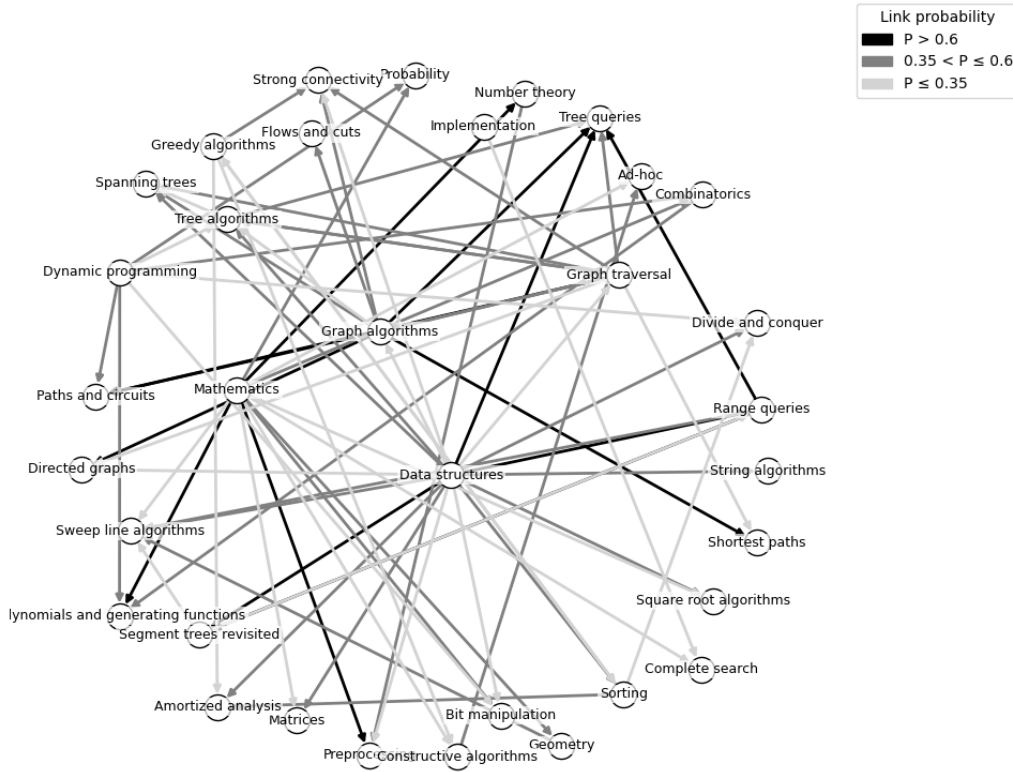


Figure 12: Conditional Probabilities in TACO

This graph provides a useful first approximation of the prerequisite structure of the domain. In particular, more foundational themes such as mathematics or data structures tend to have many outgoing edges, suggesting that they should be mastered before more specialized topics. However, this construction also has clear limitations. For example, we obtain

$$\mathbb{P}(\text{problem contains graph-algorithms} \mid \text{problem contains graph-traversals}) = 53\%.$$

Does this mean that 47% of graph traversal problems can be solved without knowing what a graph is ? Of course not! In practice, problem authors often omit very basic prerequisite tags

because they are considered implicit. A graph traversal problem may not be explicitly tagged with *graph-algorithms*, even though understanding graphs is obviously necessary to solve it.

More generally, this highlights the main weakness of the approach: the resulting graph depends heavily on the completeness and consistency of the tags attached to problems. Conditional probabilities can reveal useful patterns, but they remain sensitive to annotation bias and therefore do not always reflect true pedagogical dependencies.

3.2 KNOWLEDGE GRAPH FROM AN LLM

Because of these limitations, we explored a second method for constructing prerequisite graphs, this time relying on a large language model rather than solely on co-occurrence statistics. The goal was again to produce a graph that could serve as input to the framework developed in Part 2, but using semantic and pedagogical judgments rather than tag frequencies alone.

Instead of asking the model to generate a graph in a single step, we designed an *agentic workflow* in which the model successively proposes, justifies, critiques, and evaluates prerequisite edges between knowledge components. This decomposition was intended to make the construction process more transparent and to reduce the number of weak or poorly motivated dependencies.

The process starts from a manually curated set of nodes, corresponding to knowledge components relevant to competitive programming. These nodes were collected from several complementary sources: topic taxonomies found on educational websites such as GeeksforGeeks, as well as our own experience in competitive programming and ICPC preparation. We then retained only the concepts that were pedagogically meaningful for our setting.

Given this node set, we ask the LLM to generate edges under the constraint that the final graph should represent a prerequisite structure, that is, a directed acyclic graph (DAG). The workflow is composed of four successive stages. In the first stage, the model proposes all plausible *direct* prerequisite edges between the nodes. In the second stage, it justifies each proposed edge by explaining why the source skill is required before the target skill. In the third stage, the model adopts an adversarial role and challenges every proposed edge, for instance by arguing that the dependency is too weak, incorrectly oriented, or simply transitive rather than direct. Finally, in a fourth *judge* stage, the model decides for each edge whether it should be kept or discarded, assigns it a confidence score, and explains its decision by weighing the arguments from the defense and critique stages.

This adversarial construction procedure has two main advantages. First, it encourages the model to explicitly reflect on each dependency instead of producing a graph in a single opaque step. Second, it yields richer metadata for every edge, including a justification, a counter-argument, and a confidence score, which makes the resulting graph easier to inspect and analyze. The confidence score provides a natural control parameter: by selecting a threshold between 0 and 1, one can choose to retain only the most reliable dependencies, or alternatively keep a denser graph for exploratory analysis.

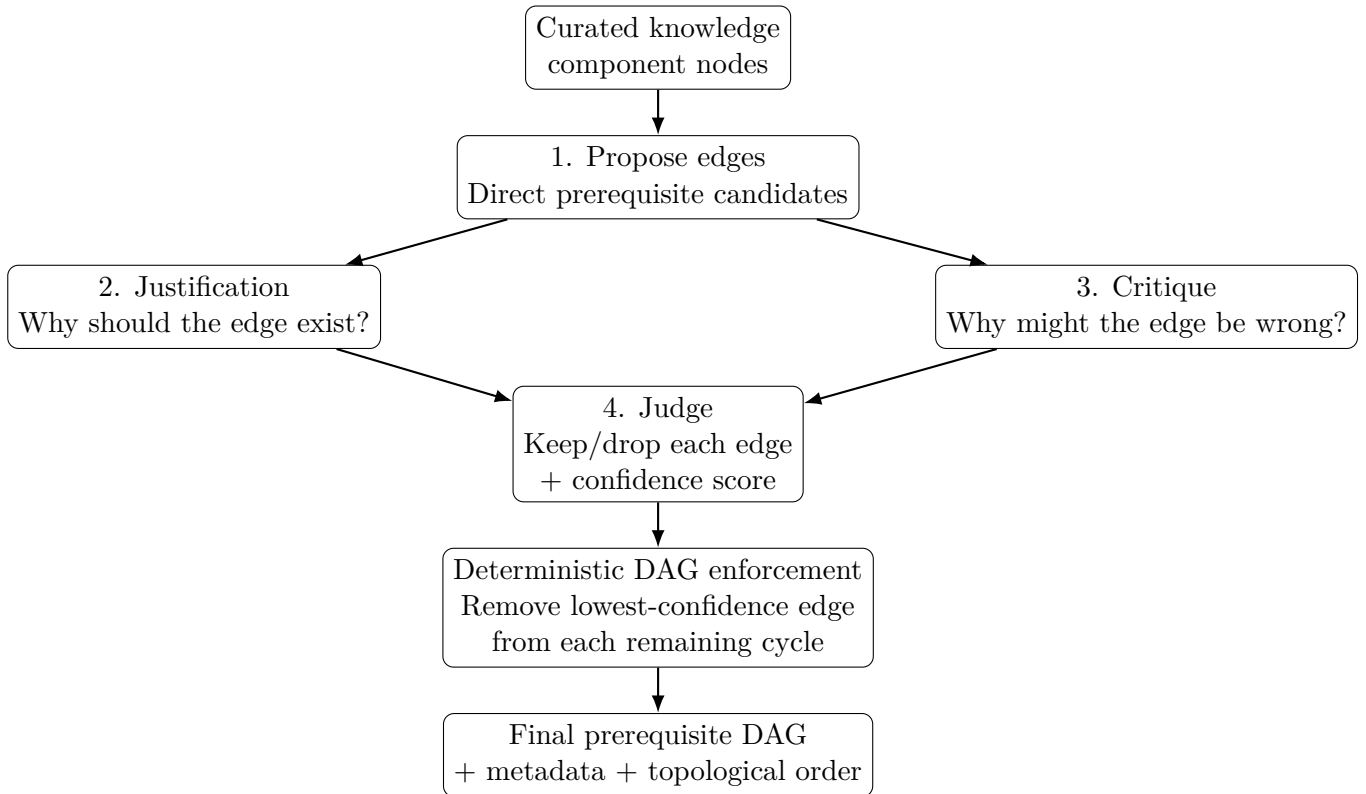


Figure 13: LLM-based adversarial workflow used to construct the prerequisite knowledge graph. Arrows indicate how intermediate outputs are passed between stages.

Because LLM outputs do not strictly guarantee acyclicity, we add a final deterministic post-processing layer. If the graph returned by the model still contains cycles, we repeatedly remove the edge with the lowest confidence score in each detected cycle until the graph becomes acyclic. This ensures that the final output always satisfies the DAG constraint required by our prerequisite-based interpretation.

We implemented this pipeline in Python with the **LangGraph** framework, using **GPT-5-mini** as the underlying language model. Given a flat list of knowledge components as input, the system outputs a prerequisite DAG enriched with edge-level metadata, along with a topological ordering of the concepts. We also developed a visualization tool that allows us to inspect the generated graph, trace the successive stages of the workflow, and manually assess the coherence of the retained dependencies.

We used 46 knowledge components. An interactive version of the graph, where each node can be clicked to display additional details, is available on this page: [link](#). Overall, the result appears quite coherent. For instance, the graph correctly captures that graphs are a prerequisite for DFS, which is itself a prerequisite for topological sorting.

Once such prerequisite graphs are available, they can be used as structured inputs for the methods developed in Part 2. In particular, they make it possible to reason about a learner’s

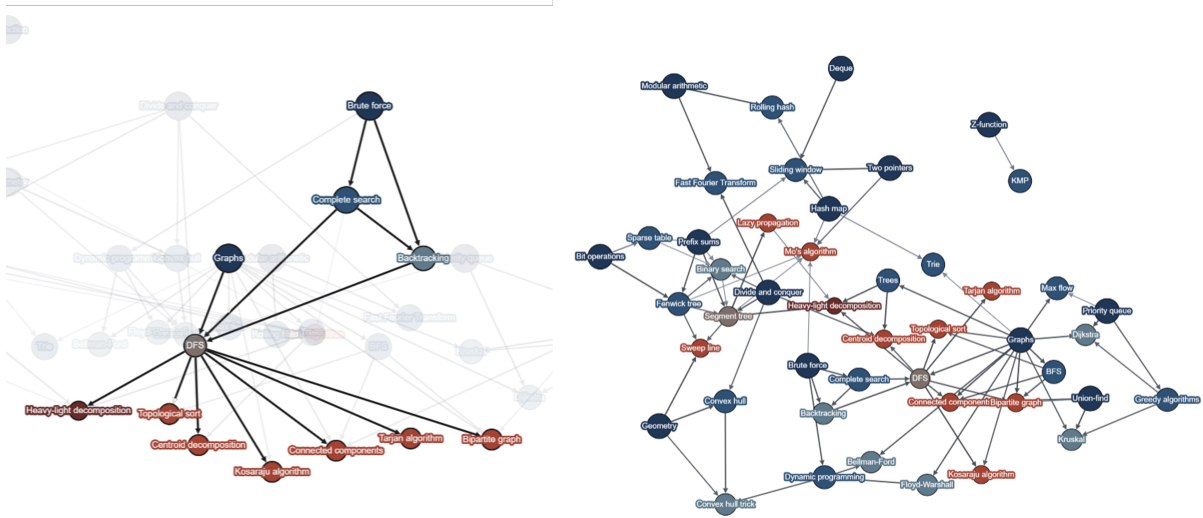


Figure 14: Prerequisite graph for competitive programming generated using the LLM-based adversarial workflow. The left panel shows only the neighbors of DFS. Colors indicate depth in the graph, with red corresponding to the greatest depth.

knowledge state, identify concepts that are likely mastered or missing, and organize recommendations in a pedagogically coherent order. This motivated us to go one step further and build a practical tool capable of analyzing a user’s competitive programming profile and turning this analysis into concrete problem recommendations.

3.3 CFTracker : KNOWLEDGE STATE ANALYSIS AND PROBLEM RECOMMENDATION

To put these ideas into practice, we developed *CFTracker*, a web application designed as a competitive programming assistant tailored to a user’s Codeforces profile. Codeforces is one of the most widely used competitive programming platforms, where users solve algorithmic problems and participate in rated contests.

CFTracker serves as a practical complement to the previous parts of the report. Along the way, its development also gave us the opportunity to experiment again with several additional recommendation and profiling models.

We chose to base this tool on Codeforces for several reasons. The platform has a large and active user base, provides a relatively accessible API, and is a setting with which we are personally familiar. It therefore offered a natural testbed for experimenting with profile analysis and recommendation methods.

3.3.1 • EXPLORATORY RECOMMENDATION BASELINES

One direction we explored was to recommend problems to a user based on what other, similar users found challenging. We used Mangaki [23], a recommendation system for mangas co-created by our tutor Jill-Jênn Vie, as a model. Mangaki recommends manga to a user by analysing what manga they liked or disliked and comparing it to the preferences of users similar to them.

Mangaki combines three recommendation approaches, which we adapted to the competitive programming setting. The first, and simplest, is K -nearest neighbors (KNN), which identifies the users most similar to a target user based on their submissions. The second is a matrix factorization approach: users and problems are represented by two matrices, U and V , and the probability that user i succeeds on problem j is modeled by the product UV , with entry (i, j) given by $U_i V_j$. We optimized this model using stochastic gradient descent. The third approach incorporates predefined tags. It replaces the problem matrix V by a tag matrix T , where each row describes the tags associated with a problem, and uses LASSO to estimate a user representation U such that $UT \approx R$. In this setting, U_i can be interpreted as user i 's affinity with each tag. While Mangaki relies on tags inferred automatically from manga covers, in our case we can directly use the tags provided with Codeforces problems.

First, for KNN, we first needed to define a similarity function. Our first choice of metric was a cosine similarity between the submission verdicts. As users rarely attempt problems much too difficult for them, we also tried cosine similarity between submissions (value of 1 if both users have a submission for that problem, 0 otherwise).

Table 3: Spread and Gap for both similarity functions. Gap = Mean Neighbour Elo - User ELO; Spread = Std of Neighbour Elo

Similarity	Mean Spread	Std Spread	Mean Gap	Std Gap	Min Gap	Max Gap
Cosine (Verdict)	304.6	93.4	+244.2	233.2	-377.4	+1264.5
Cosine (Submission)	317.2	103.3	+245.6	245.5	-320.1	+1322.9

Several observations emerge from these results. First, the difference in results between both metrics is very slim. This was to be expected, as most submissions correspond to successful attempts. More importantly, KNN tends to retrieve neighbors with a substantially higher Elo than the target user. In some cases, the difference exceeds 1200 Elo points. A plausible explanation is that stronger users are also more active and therefore more likely to overlap with weaker users on easier problems. We also observe both a large mean gap and a large spread, suggesting that the neighborhoods found by KNN are relatively heterogeneous. This difficulty is likely linked to the extreme sparsity of the data. Our experiments used all submissions from 1,685 active users, corresponding to 2,876,458 submissions over 29,360 problems. For a random

(user, problem) pair, there is therefore more than a 94% chance that the user never attempted the problem.

For matrix factorization with stochastic gradient descent, we tried varying the amount of components, that is the number of components for each user and each problem in order to find an optimum.

Table 4: Mean Absolute Error and Area Under Curve for matrix factorization based on the number of components (80 iterations)

Dimension	MAE	AUC
1	0.4550	0.7167
4	0.4547	0.7176
16	0.4549	0.7176
64	0.4546	0.7182
256	0.4544	0.7182

We notice that the error is large, and the AUC is rather mediocre. What’s quite surprising is how little our metrics vary even when we drastically increase the number of components. This would suggest that there is one core dimension for measuring a users skill, which would be an argument in favor of linear models like elo.

For LASSO, we quickly encountered a major limitation: problem tags encode topic information, but not difficulty. As a result, applying LASSO directly to the tag vectors yields predictions that are close to random. To partially address this issue, we introduced an additional “Elo” tag, scaled between 0 and 1 and proportional to the problem’s difficulty rating.

Table 5: Mean Absolute Error and Area Under Curve for LASSO

Model	MAE	AUC
Standard LASSO	0.4594	0.5985
LASSO with ELO	0.4532	0.6071

As we see, LASSO is barely better than random guessing, its predictions are very weak. The addition of an ELO tag is only a slight upgrade, as its importance is drowned by the fact that there are many other tags. We can conclude that this algorithm may be good to determine user preferences, but it is absolutely unable to handle difficulty and as such completely unfit for our situation. We might however be able to use it for a problemset that only contains problems of similar difficulty, but we currently lack the data to prove it.

Overall, these experiments suggest that recommendation methods based on user similarity or latent preferences are less effective in this setting than one might hope. Competitive programming differs from domains such as cultural recommendation because performance is dominated by a strong notion of difficulty: users who can solve difficult problems can usually also solve easier ones, independently of their preferences. This hierarchical structure is not naturally captured by KNN, standard matrix factorization, or LASSO.

3.3.2 • FINAL RECOMMENDATION MODEL: AUGMENTED ELO

These observations led us back to a simpler but more robust approach based on rating models. Our final solution consisted in extending the classical Elo framework by introducing skill-specific ratings. Instead of assigning a single rating to each user, we associated separate ratings with several competitive programming topics, identified through problem tags. Concretely, we selected 15 major skills and computed, for each user, a tag-specific Elo score intended to reflect their level in that area.

This decomposition revealed that a user's profile is often far from one-dimensional. In many cases, the gap between the strongest and weakest skill-specific ratings exceeded 200 Elo points. This observation supports the idea that a single global rating is too coarse to capture the diversity of a learner's strengths and weaknesses.

We then used these tag-specific ratings to build a domain-aware recommender. Given a particular tag, the system suggests a problem whose estimated difficulty is close to the user's Elo in that area. This makes it possible to recommend exercises that are adapted not only to the user's overall level, but also to their topic-specific competence.

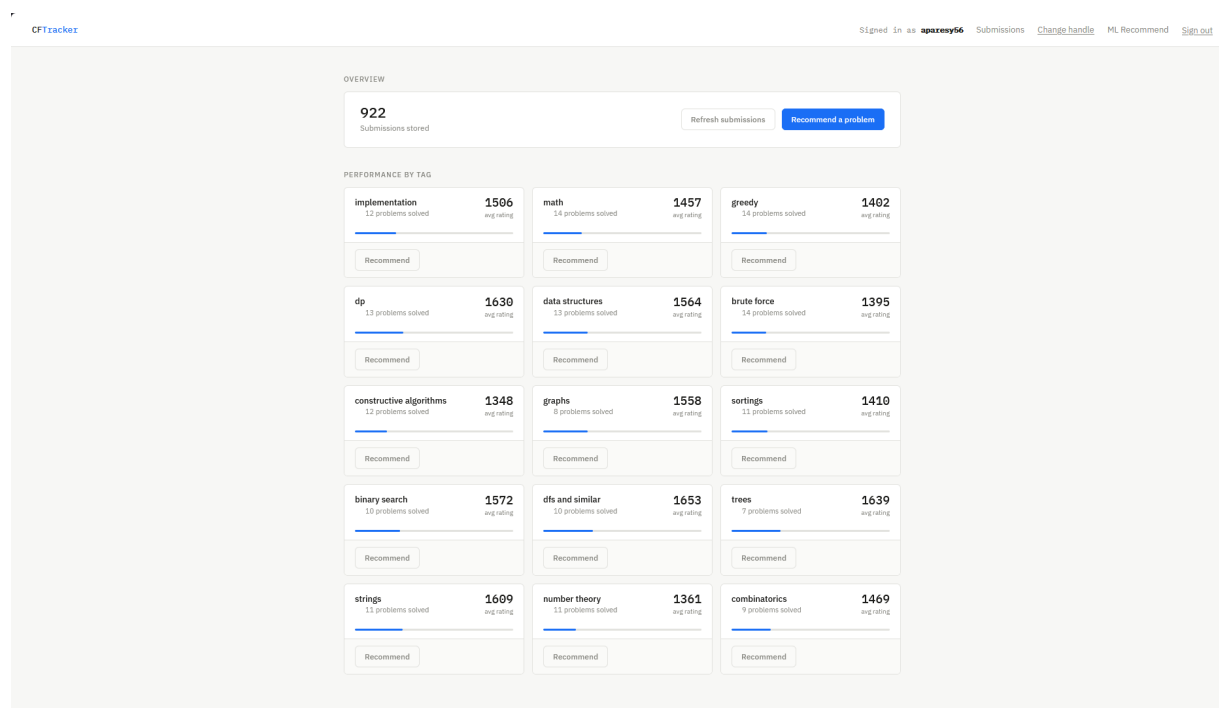


Figure 15: Tag-based-ELO problem recommender. It recommends a problem adapted to a tag and the user's competence in the associated domain.

This version of CFTracker is interpretable and efficient. It provides users with a finer-grained profile than the standard Codeforces rating and directly turns this profile into personalized recommendations.

4

PROJECT EVALUATION AND PROGRESS REVIEW

4.1 ACHIEVEMENT OF OBJECTIVES

In our first definition of project goals, we aimed at achieving two main objectives : experimenting with and generating representations of knowledge states, and creating tools to help students. We find that both of these objectives were fulfilled in different ways. For the first objective, we mainly explored the representations of knowledge states through knowledge graphs, and we demonstrated that in particular dependency graph were very useful for the task of knowledge state representation. For the second objective, we managed to code a website for problem recommendation in competitive programming, which is one of the main ideas that we had in mind since the start of the PSC. Even though this website could be improved by applying more techniques extracted from the work that we did in the PSC, we are still satisfied with the outcome as we were not sure that we could create a website as part of the PSC.

We found that writing a preprint for AIED as well as taking a closer look to dependency graphs (cf. part 3) allowed us to expand our research topic in ways that we did not think of at first, while still working with notions related to our original subject. Therefore, this helped us complete another one of our objectives, which was to immerse ourselves in hands-on research in order to have a first experience.

4.2 FURTHER EXPLORATION POSSIBILITIES

Some topics were quickly mentioned in our initial report or during our meetings but were not studied as much as we would have liked to, thus we present the ones that could lead to the most interesting expansions to our project. First, it would have been possible to study in depth other methods to determine the knowledge state of a student, in different contexts. Second, the exploration of standard knowledge tracing models other than the ones that we studied as part of the PSC could have lead us to some other ideas in order to improve every part of our work. Lastly, the implementation of a complete agent system in order to setup an interactive tutoring application would have been interesting from a practical standpoint, and could have lead to testing in a real-world setting where students could see improvement over time using our tool.

4.3 EXPOSURE TO RESEARCH

As mentioned previously, one of the main benefits that we extracted from committing to this project is the experience that we gained in this setting. Not only did the PSC allow us to learn searching for a proper resolution path to such a broad problem as intelligent tutoring, but we were also taught the intricacies in the process to create a paper exposing our contributions. As such, this project greatly helped us in getting our personal standpoints on the topic of scientific research, and allowed us to think about whether we would like this work to be similar to our careers later on.

4.4 GROUPWORK

Teamwork played a central role in the success of our project. Although we initially planned to divide the work strictly among group members, we quickly realized the benefits of a more collaborative and flexible approach. Each member contributed beyond their assigned tasks, offering support, feedback, and insights across different aspects of the project. This allowed us to maintain a coherent vision and ensured that knowledge was shared within the team.

Regular discussions and meetings helped us coordinate our efforts, address challenges collectively, and refine our ideas. This collaborative dynamic not only improved the overall quality of our work but also strengthened our ability to communicate and adapt within a research-oriented environment. As a result, the groupwork experience was both productive and enriching, providing us with valuable skills for future academic and professional projects.

CONCLUSION

This PSC allowed us to explore in depth the question of how machine learning and structured knowledge representations can be used to support the learning of programming. Starting from this broad objective, we investigated three complementary directions. First, we studied knowledge tracing models, from classical approaches such as Elo to more advanced deep learning methods such as AKT, before extending this work to the liveKT setting, which resulted in a paper accepted at AIED 2026. Second, we developed a theoretical framework for dynamic cognitive diagnosis based on prerequisite graphs, order ideals, and adaptive querying strategies. Third, we turned to practical applications by constructing knowledge graphs for competitive programming and developing *CFTracker*, a tool for profile analysis and problem recommendation.

Beyond these concrete results, the project gave us a valuable first experience of research. It required us to read and connect ideas from the literature, formulate problems precisely, implement and evaluate models, and reflect critically on both the strengths and limitations of our approaches. It also showed us how theoretical, experimental, and practical perspectives can complement one another within the same project.

Overall, we believe that we achieved the main objectives we had set at the beginning of the PSC, while also opening several promising directions for future work. In particular, integrating the different components we developed into a more unified tutoring system, making the platform available to a wider audience, further exploring the use of LLM-based agents, and extending our diagnosis models to more realistic settings all appear as natural continuations. More broadly, this project confirmed for us that intelligent tutoring systems for programming constitute a rich and meaningful research area, at the intersection of machine learning, education, and algorithmic reasoning.

REFERENCES

- [1] Zhenya Huang, Yonghe Zheng, Minghao Yin, Minjuan Wang, Enhong Chen, Shuanghong Shen, Qi Liu. A survey of knowledge tracing: Models, variants, and applications. *IEEE Transactions on Learning Technologies*, 2024.
- [2] Philip I Pavlik Jr, Hao Cen, and Kenneth R Koedinger. Performance factors analysis—a new alternative to knowledge tracing. *Online submission*, 2009.
- [3] Margit Antal. On the use of elo rating for adaptive assessment. *Studia Universitatis Babes-Bolyai, Informatica*, 2013.
- [4] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. *Advances in neural information processing systems*, 28, 2015.
- [5] Aritra Ghosh, Neil Heffernan, and Andrew S. Lan. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2330–2339, New York, NY, USA, 2020. Association for Computing Machinery.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [7] Mounir Lbath, Alexandre Paresy, Abdelkayoum Kaddouri, Abdelrahman Zighem, and Jill-Jênn Vie. Live knowledge tracing: Real-time adaptation with tabular foundation models. *arXiv preprint arXiv:2602.06542*, 2026.
- [8] Indronil Bhattacharjee and Christabel Wayllace. Cold start problem: An experimental study of knowledge tracing models with new students. In *International Conference on Artificial Intelligence in Education*, pages 425–432. Springer, 2025.
- [9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [10] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. Tabicl: A tabular foundation model for in-context learning on large data. In *ICML 2025-Forty-Second International Conference on Machine Learning*, 2025.
- [11] Jake Robertson, Noah Hollmann, Samuel Müller, Noor Awad, and Frank Hutter. Fairpfn: A tabular foundation model for causal fairness. In *Forty-second International Conference on Machine Learning*.

- [12] Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. Tabpfn: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations (ICLR)*, 2023.
- [13] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.
- [14] Alexander Pfefferle, Johannes Hog, Lennart Purucker, and Frank Hutter. nanotabpfn: A lightweight and educational reimplementation of tabpfn. In *EurIPS 2025 Workshop: AI for Tabular Data*.
- [15] Mingyu Feng, Neil Heffernan, and Kenneth Koedinger. Addressing the assessment challenge with an online system that tutors as it assesses. *User Modeling and User-Adapted Interaction*, 19(3):243–266, 2009.
- [16] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [17] Zitao Liu, Qiongqiong Liu, Jiahao Chen, Shuyan Huang, Jiliang Tang, and Weiqi Luo. pykt: a python library to benchmark deep learning based knowledge tracing models. In *Advances in Neural Information Processing Systems*, volume 35, pages 18542–18555, 2022.
- [18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [19] Jean-Claude Falmagne, Mathieu Koppen, Michael Villano, Jean-Paul Doignon, and Leila Johannesen. Introduction to knowledge spaces: How to build, test, and search them. *Psychological Review*, 97(2):201, 1990.
- [20] Nathan Linial and Michael Saks. Every poset has a central element. *Journal of Combinatorial Theory, Series A*, 40(2):195–210, 1985.
- [21] J. Scott Provan and Michael O. Ball. The complexity of counting cuts and computing the probability that a graph is connected. *SIAM Journal on Computing*, 12(4):777–788, 1983.
- [22] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. Taco: Topics in algorithmic code generation dataset. 2023.
- [23] Mangaki. Research @ mangaki. <https://research.mangaki.fr/>.